

Rapport 2

Systèmes d'Exploitation – NACHOS

Devoir 2 : Gestion des threads utilisateurs

Mervyn Samsan - Robert Dylan - Negre Damien

9 novembre 2025

I. Bilan

- L'objectif de ce TP était d'implémenter la gestion des threads utilisateurs dans NachOS, à travers les appels systèmes `ThreadCreate` et `ThreadExit`.
- Nous avons réussi à faire fonctionner ces appels systèmes, aussi bien côté noyau que côté utilisateur.
- Les threads peuvent désormais s'exécuter en parallèle dans le même espace mémoire, sans interférer entre eux.
- Chaque thread dispose de sa propre pile grâce aux fonctions `AllocateUserStack` et `FreeUserStack`, ce qui évite les conflits entre variables locales.
- La terminaison du processus est correctement gérée : le programme ne s'arrête que lorsque le dernier thread actif appelle `ThreadExit`, grâce à un compteur de threads dans `l'AddrSpace`.
- Nous avons également implémenté la terminaison automatique en plaçant l'adresse de `ThreadExit` dans le registre `$ra` lors de la création du thread, ce qui assure un arrêt propre.
- Enfin, nous avons également implémenté la dernière partie concernant les sémaphores utilisateurs. Notre test producteur-consommateur est fonctionnel et démontre une synchronisation correcte, s'exécutant en boucle continue et affichant la succession attendue de 'P' (Producteur) et 'C' (Consommateur).

II. Points délicats

- Ce TP a été plus complexe que le premier, surtout pour bien comprendre la gestion du contexte d'exécution et de la mémoire dans NachOS.
- La fonction `StartUserThread` a été la partie la plus difficile à mettre en place. Il fallait créer un nouveau contexte pour le thread tout en gardant le même espace mémoire que le thread principal.
- Le passage des paramètres a également été un point sensible. Comme `Thread::Start` ne prend qu'un seul argument, nous avons créé une structure `UserThreadParams` regroupant plusieurs informations : l'adresse de la fonction à exécuter, son argument, l'adresse de sortie (`exitAddr`) et le sommet de la pile (`stackTop`). Cette structure est allouée dans `do_ThreadCreate` puis libérée au début de `StartUserThread`.
- Il fallait ensuite :
 - Allouer une pile spécifique pour le thread via `AllocateUserStack()` ;
 - Initialiser tous les registres MIPS à zéro pour éviter les comportements imprévisibles ;

- Configurer correctement les registres importants : le `PCReg`, le `NextPCReg`, le `StackReg`, le registre d'argument (`$a0`) et le registre d'adresse de retour (`$ra`).

- La moindre erreur dans ces registres provoquait un crash du programme, ce qui a rendu le débogage assez long.

Pour corriger les erreurs, nous avons d'abord ajouté des `printf` et des `PutChar` pour suivre le déroulement du code. Notre chargé de TD nous a ensuite expliqué la logique à adopter pour localiser précisément l'origine des erreurs, en nous montrant notamment comment analyser le déroulement du contexte et des registres. Nous avons également utilisé les options de débogage intégrées à NachOS (notamment `-d` et `-d t`), qui se sont révélées très utiles, non seulement à ce moment-là, mais aussi plus tard dans le projet pour comprendre et résoudre d'autres problèmes.

III. Limitations

- Notre implémentation fonctionne correctement, mais présente encore quelques limites malgré l'ajout des sémaphores utilisateurs pour la synchronisation.
- Cela implique que si plusieurs threads accèdent à une même variable en même temps, des conditions de course peuvent apparaître.
- Le système devient également instable si trop de threads sont créés, car la taille du `Bitmap` gérant les piles est fixe.

IV. Tests

Tous nos tests ont été réalisés dans le fichier `makethreads.c`, que nous avons utilisé pour valider chaque étape de notre implémentation.

- **Test de base** : Nous avons commencé par créer un seul thread affichant un caractère pendant que le `main` restait dans une boucle infinie. Cela nous a permis de vérifier le bon fonctionnement de `ThreadCreate` et `StartUserThread`.
- **Test de concurrence** : Nous avons ensuite lancé plusieurs threads affichant des messages différents (par exemple "AAAAA" et "BBBBB"). Le mélange des sorties à l'écran nous a confirmé que les threads s'exécutaient bien en parallèle.
- **Test de stress** : Nous avons créé 3 threads, chacun affichant deux lettres 'a'. Nous avons bien obtenu au total 6 caractères 'a' à l'écran, ce qui prouve que tous les threads

se sont exécutés correctement et indépendamment. Aucun crash n'a été observé, ce qui confirme la bonne gestion des piles et du partage de mémoire.

- **Test de terminaison** : Enfin, nous avons vérifié que le programme ne se termine qu'après la fin du dernier thread actif. Après que tous les threads enfants aient appelé `ThreadExit`, le processus principal s'arrêtait proprement.

Ces différents tests nous ont permis de valider le bon fonctionnement global de la gestion des threads utilisateurs dans NachOS.